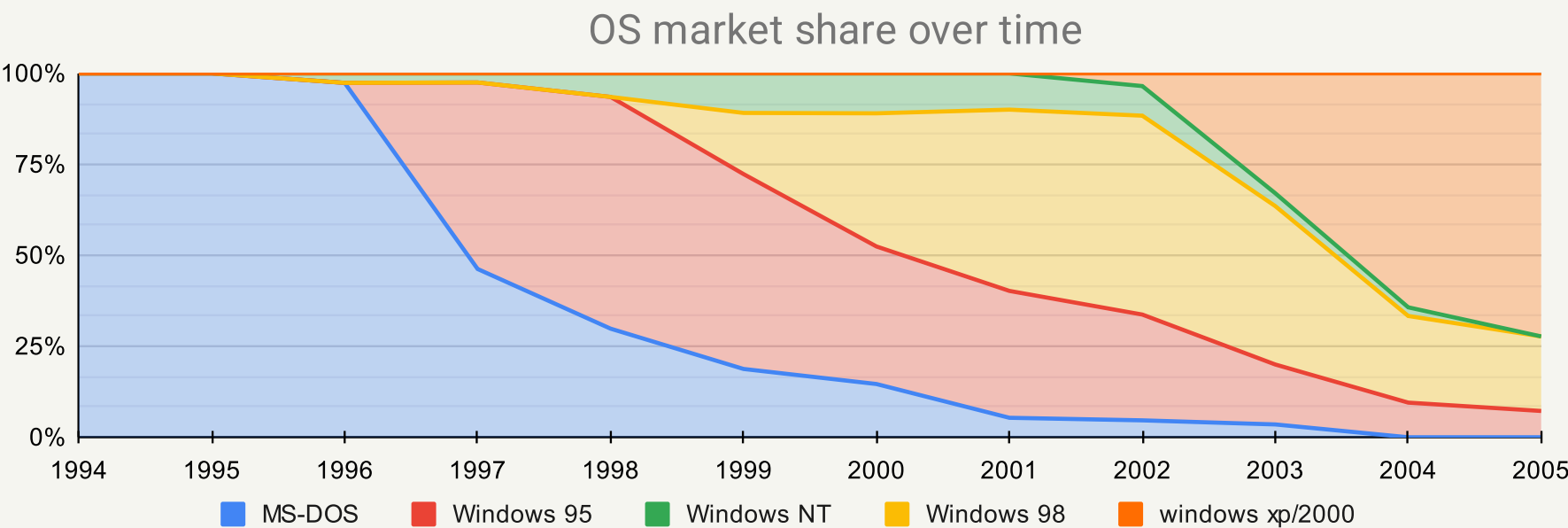


Nov 17, 2025

HOW QUAKE.EXE GOT ITS TCP/IP STACK

Released in June 1996, *Quake* had to ride three technological shock-waves during its lifetime. Besides the emergence of 3D hardware accelerator cards and the growth of the Internet, an operating system shift put game developers in a tough position.



With its push for Windows 95 and Windows NT, Microsoft was replacing its legacy PC operating system, MS-DOS. From 1996 to 1997, the market share of DOS dropped by 50%. Some developers, like Blizzard North, took the leap of faith and wrote Windows 95-exclusive titles such as *Diablo*. id Software on the other hand went through the effort of producing a single binary, *quake.exe*, able to run on both DOS and Windows.

What is even more impressive is that id managed to make *Quake* better when Windows 95 TCP/IP stack was available. Here is how they did it.

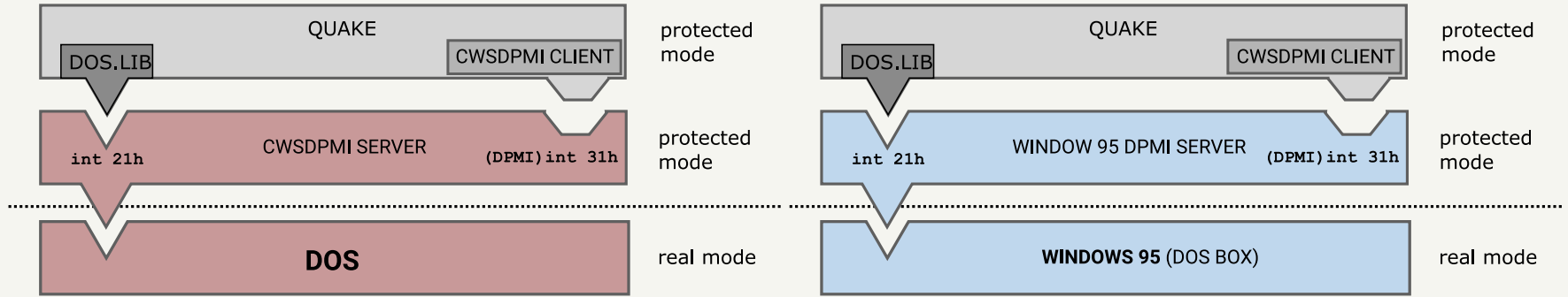
QUAKE.EXE 101

*quake.exe* is a DOS executable. id Software had used Watcom compiler for DOOM but they switched to a GCC port named *djgpp*<sup>[1]</sup> to cross-compile *Quake* on Alpha servers.

```
$ file quake.exe
quake.exe: MS-DOS executable, COFF for MS-DOS, DJGPP go32 DOS extender
```

Alike watcom's DOS/4GW, *djgpp* offered to developers an extender allowing to write programs with flat 32-bit addressing instead of the dreaded 16-bit near/far hellish real-mode otherwise mandated by DOS. An extender works with a client and a server. In the case of *Quake* the extender client is embedded in *quake.exe* while the server is in *cwsdpmi.exe*.

From the beginning of the development, id had requested from *djgpp* engineers that their DPMI client would be able to run on *djgpp*'s DPMI server but also Windows 95 DPMI server.



It may not be apparent how much of a tour-de-force it was for *djgpp* to make their DPMI client work with another DPMI server but knowing a little about how it works, it blows me away. Raymond Chen, Microsoft kernel engineer at the time, had the best description of how to perceive this situation.

*The client application was written with the assumption that it is using the MS-DOS extender that is included with the application, but in reality it is talking to the DPMI host that comes with Windows.*

*The fact that programs seem to run mostly okay in spite of running under a foreign extender is either completely astonishing or totally obvious, depending on your point of view.*

*It's completely astonishing because, well, you're taking a program written to be run in one environment, and running it in a different environment. Or it's totally obvious because they are using the same DPMI interface, and as long as the interface has the same behavior, then naturally the program will continue to work, because that's why we have interfaces!*

- Raymond Chen<sup>[2]</sup>

Being able to run with Windows 95 DPMI server was how *quake.exe* pulled off the ability to run under both DOS and Windows 95.

QUAKE.EXE UNDER DOS

Doom only needed two files to run, `doom.exe` and `doom.wad` but there are many files that came with Quake.

```
$ find quake
./mgenvxd.vxd      ./genvxd.dll ./qlaunch.exe      ./id1/pak0.pak    ./pdipx.com
./cwsdpmi.exe      ./q95.bat    ./id1/config.cfg    ./quake.exe       ./quakeudp.dll
```

It looks like a mess at first sight but running Quake under DOS only requires four files. Namely, the game engine `quake.exe`, the config file `config.cfg`, the asset file `pak0.pak`, and the DOS extender server `cwsdpmi.exe`.

```
./mgenvxd.vxd      ./genvxd.dll ./qlaunch.exe      ./id1/pak0.pak    ./pdipx.com
./cwsdpmi.exe      ./q95.bat    ./id1/config.cfg    ./quake.exe       ./quakeudp.dll
```

QUAKE.EXE UNDER DOS: MULTIPLAYER

Quake supported four types of multiplayer protocols.

Two modes allowed gamers to enter a duel (1v1). Both modes expected a device plugged into the COM port of the PC. A modem allowed to call an opponent's phone number (hello \$\$\$) while a NullModem cable (called here "Direct Connect") required both computers to be a few feet apart.

Both IPX and TCP/IP allowed a much more interesting deathmatch featuring up to 16 players. IPX technology was intended for LAN where all machines were a few feet apart, while TCP/IP allowed to reach anybody worldwide.



Notice how, under DOS, by default, both IPX and TCP modes were disabled (greyed out).

QUAKE.EXE UNDER DOS: GREYED OUT MULTIPLAYER MODES

Quake came with **PDIPX.EXE** which loaded an IPX DOS TSR. That TSR communicated with a packet driver which in turn hit the network card. Quake was able to probe for that DOS TSR and upon detection allowed players to select IPX.

Using TCP/IP was nearly impossible. DOS did not come with a TCP/IP stack and it was something complex enough that only a single vendor provided a TSR for it on DOS.

The TSR name was BWNFS. Made by *Beame & Whiteside*, its cost \$395 in 1996 (\$830 in 2025!)[3]. It is reasonable to say that few gamers ever used TCP/IP on DOS to play QUAKE.



QUAKE.EXE UNDER WINDOWS 95

Starting `quake.exe` from Windows 95 works like a charm. The executable is loaded into a Windows 95 "dos-box"[4] that virtualizes memory, interrupts, and signals[5]. The game ran exactly like under DOS with the same multiplayer choices available. It was convenient since users did not have to load any mouse driver or set up the `BLASTER` environment variable to make the sound card work.

Much less convenient however, this way to run Quake requires 16 MiB RAM. Quake only needs 8 MiB but Windows 95 adds quite a bit of overhead! The same files used when running from DOS are used here as well, except for `cwsdpmi.exe`, since the DJGPP client detects and uses Windows' built-in DPMI server.

```
./mgenvxd.vxd      ./genvxd.dll ./qlaunch.exe      ./id1/pak0.pak    ./pdipx.com
./cwsdpmi.exe      ./q95.bat    ./id1/config.cfg    ./quake.exe       ./quakeudp.dll
```

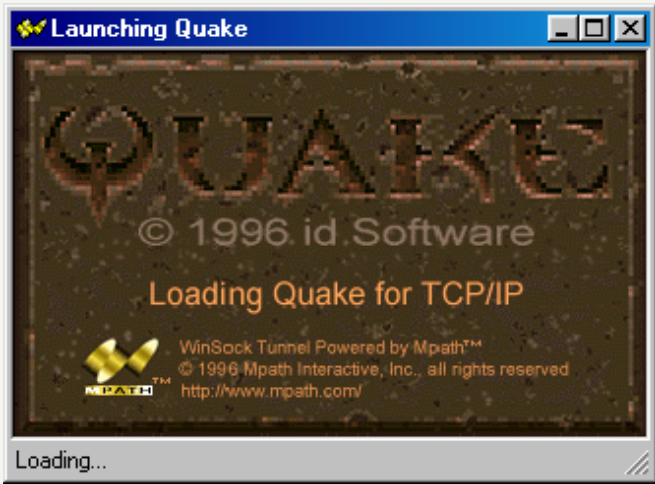
**Trivia:** It is impressive to see *Quake* run at full speed knowing that Windows 95 runs DOS executable in a virtual machine. My guess is that, in full screen, memory writes and reads to the VGA are given direct access to the hardware to preserve performances.

THE MAGICAL Q95.BAT SCRIPT

Starting `quake.exe` from DOS or Windows are not the only two options to run *Quake*. There is a third one which is to launch `q95.bat`.

In this case, a window "Launching Quake" briefly pops up on Windows 95 desktop.

The text gives a clue about what is happening. Quake is loaded with a tunnel to Winsock, Microsoft's TCP/IP stack. There is further indication of *what* is doing that, "Powered by Mpath". But not much more to explain *how* this all works.



MPATH

Mpath Interactive was a company dedicated to online gaming. They provided subscription services to help gamers find each other but also operated as an ISP reseller.<sup>[6]</sup> It was in their interest to help gaming companies to release titles allowing Internet play as Larry Hastings, an Mpath employee at the time, recalls.

*Back then in the primordial ooze that was the mid-90s internet, online multiplayer was still in its infancy. If you wanted to play a multiplayer game on the internet, either you needed to have explicit host & port information, or you needed to use an online multiplayer gaming service. And in 1995 there were only two: us, and Total Entertainment Network. You might think game creators would come to us and say "please put my game on your service!", but... nope! Not only did we have a licensing team that went out and got contracts to license games for our service, but we had to pay the vendor for the right to license their game, which was often an exclusive. So, we had Quake and Unreal; TEN got Duke Nukem 3D and NASCAR.*

*The user experience for Mplayer was like this. First, you'd run the "Gizmo", which was a Windows program that acted as a sort of game browser. It knew which compatible games you had installed, and it'd let you browse the multiplayer games on offer for each game; the metaphor we used for this was a "room". Quake was drop-in, so you could simply find a game in progress and hop right in--not a feature of very many games back then. Alternatively, you could find a "room" where someone was proposing to launch a game soon. Or you could create your own. You'd set the name of the room, and the Mplayer Gizmo had some per-game UI that let you set the settings for the game (what map, what features, etc). The room featured text and audio chat, and even a shared "whiteboard", a simple paint program. Once the owner of the "room" "launched" the game, everyone's Gizmos would automatically start the game for them, and the game would automatically join that online game and start playing.*

*In order for a game to run on Mplayer, it had to integrate with the Mplayer software stack. Mostly this integration work was done by Mpath engineers; we'd get source code from the game developer and "porting engineers" would get it to run on Mplayer. This often included modifying both the client and the server, so that both could talk via Mplayer's servers.*

*The early version of Quake was DOS only, and used the Chunnel to talk to the Windows 95 TCP/IP stack. (Which in retrospect makes the "Chunnel" a type of "thunk", like Microsoft's "Win32s".) I think the deal was, we licensed the Chunnel to id, and in return for that we got to have Quake on Mplayer. So, DOS Quake supported running on Mplayer via the Chunnel, in addition to connecting to open game servers on the Internet via host and port.*

*- Larry Hastings (Email conversation)*

Larry was kind enough to share some Quake anecdotes.

*One afternoon shortly after we got our first build of the game, we played a round of deathmatch with the id team over the internet. We were in Cupertino, CA, in a building on Bandle Drive (now a "Fitness Center" for Apple employees). They of course were in Mesquite TX. Yup, it was deathmatch over the internet--very exciting!*

*The only id employee I remember for sure being in the game was Tim Willits. He owned us, both because he was way more used to Quake, but also because he knew where all the secrets were. At one point I spotted him coming out of a secret doorway with a rocket launcher. And either he didn't see me, or I died shortly thereafter.*

*- Larry Hastings (Email conversation)*

As for explaining how the Chunnel worked, I was out of luck.

*I didn't work on the Chunnel. That was mainly a British guy named Henry but I don't remember his last name, it was thirty years ago. All I remember about him is what he looked like, and the fact that he drove a cool car, a white Merkur XR4Ti.*

*- Larry Hastings (Email conversation)*

## GHIDRA

When everything else fails, we still have Ghidra and doomworld's amazing community (thanks xttl<sup>[7]</sup>). After much decompiling and talking, it turned out all files previously ignored were part of Mpath's "Chunnel".

|                            |                           |                               |                             |                             |
|----------------------------|---------------------------|-------------------------------|-----------------------------|-----------------------------|
| <code>./mgenvxd.vxd</code> | <code>./genvxd.dll</code> | <code>./qlaunch.exe</code>    | <code>./idl/pak0.pak</code> | <code>./pdipx.com</code>    |
| <code>./cwsdpmi.exe</code> | <code>./q95.bat</code>    | <code>./idl/config.cfg</code> | <code>./quake.exe</code>    | <code>./quakeudp.dll</code> |

q95.bat is just a small script to launch mpath's main program. qlauncher.exe contains all the **MPlayer functions**. However the role of this executable is limited.

It merely loads quakeudp.dll. Despite its confusing name, this DLL is the heart of Quake Chunnel. It is the bridge to Microsoft TCP/UDP/IP stack (wsock32.dll). It also starts Quake with -path parameter to make it load a BSD network socket API sys/socket.h. Finally, it also loads the virtual device driver manager genvxd.dll.

The virtual device is the trick that allows a DOS executable running inside a Windows 95 dos box to communicate with win32. The genvxd.dll dynamic library loads a virtual device driver<sup>[8]</sup> named GENVXD.VXD which installs itself to respond on interrupt 0x48.

The last piece of the puzzle is on Quake side. The implementation of BSD sys/socket.h, mpplc.c, is code provided by Mpath. It takes care of marshaling every BSD socket function call, then use the DPMI client to trigger a software interrupt that is received in win32 land. Data is passedup the pipeline we previously described until it is unmarshalled by genvxd.dll and routed towards wsock32.dll. Notice the symmetry of **functions** found in mplib.c marshalling and the **symbols** found in genvxd.dll unmarshalling.



**Trivia:** It seems John Cash was involved in compiling Mpath's stuff. We can find his name in the symbols of mgenvxd.vxd.

F:\cashcode\GENVXD\bin\Mgenvx

**Trivia:** The source code of mgenvxd.vxd, genvxd.dll, qlaunch.exe and quakeudp.dll was never released. It was a proprietary, patented technology from Mpath. It is likely id only got permission to release the client side of it.

As far as I understood it, that is how Quake was able to send TCP and UDP packets over IP. This convoluted construct became obsolete when id stopped shipping DOS executable (the last one being vquake.exe). After Dec 1996, winquake.exe, glquake.exe, and all QuakeWorld binaries were win32 exclusive with direct access to wsock32.dll.

REFERENCES

- ^ [1] [Why did ID choose DJGPP for Quake?](#)
- ^ [2] [Getting MS-DOS games to run on Windows 95](#)
- ^ [3] [Mac and PC TCP/IP FAQ](#)
- ^ [4] [Microsoft Windows Chicago Reviewer's Guide, p69](#)
- ^ [5] [Writing Windows VxDs and device drivers](#)
- ^ [6] [MPlayer.com](#)
- ^ [7] [How did Quake work on Windows 95?](#)
- ^ [8] [Virtual Device Drivers—What is a VxD?](#)

\*

